
Методологический отчёт

Доценко Варвара, Хватов Сергей

1 Постановка задачи

Необходимо построить динамическую атрибутированную экономическую сеть на основе предоставленных данных, где каждый узел это муниципальное образование (МО), ребра отражают степень экономической близости МО. Сеть строится ежемесячно.

2 Данные

Мы использовали датасеты из двух источников:

- **СберИндекс** — Потребительские безналичные расходы на уровне муниципальных образований по категориям трат (продовольствие, медицина и т.д.)
- **Росстат**:
 - Оценка численности населения на 1 января текущего года
 - Среднесписочная численность работников организаций (без субъектов малого предпринимательства)
 - Среднемесячная заработная плата работников организаций (без субъектов малого предпринимательства)

3 Предобработка данных

Данные в датасете СберИндекса есть только за 2023-2024 год, так что мы будем строить сеть именно на этих годах.

Преобразовываем накопительные показатели в значения отдельных месяцев

Сопоставляем названия МО, которые отличаются в разных датасетах

Объединяем датасеты по региону, муниципальному образованию, году и месяцу.

Чтобы считать доли категорий трат, делим траты в различных категориях на "Все категории".

$$\text{category_share} = \frac{\text{расходы категории}}{\text{расходы «Все категории»}}.$$

Чтобы считать долю городского населения, делим количество городского населения на количество населения.

$$\text{urban_share} = \frac{\text{городское население}}{\text{население}}.$$

Разделяем виды деятельности из ОКВЭД 2 на 3 категории – промышленность, сельское хозяйство и сфера услуг и считаем долю работников в этих сферах в общем числе работников.

К промышленности относим разделы В–Е: добыча полезных ископаемых; Обрабатывающие производства; обеспечение электрической энергией, газом и паром, кондиционирование воздуха; водоснабжение, водоотведение, организация сбора и утилизация отходов, деятельность по ликвидации загрязнений

К сельскому хозяйству относим раздел А — Сельское, лесное хозяйство, охота, рыболовство и рыбоводство.

К сфере услуг относим разделы F–S: строительство; торговля оптовая и розничная, ремонт автотранспортных средств и мотоциклов; транспортировка и хранение; деятельность гостиниц и предприятий общественного питания; деятельность в области информации и связи; деятельность финансовая и страховая; деятельность по операциям с недвижимым имуществом; деятельность профессиональная, научная и техническая; деятельность административная и сопутствующие дополнительные услуги; государственное управление и обеспечение военной безопасности, социальное обеспечение; образование; деятельность в области здравоохранения и социальных услуг; деятельность в области культуры, спорта, организации досуга и развлечений; предоставление прочих видов услуг.

Формально:

$$\text{share}_s = \frac{\sum_{d \in \mathcal{D}_s} \text{численность работников в разделе } d}{\text{общее число работников}},$$

$$s \in \{\text{промышленность, сельское хозяйство, услуги}\},$$

где $\mathcal{D}_{\text{промышленность}} = \{B, C, D, E\}$, $\mathcal{D}_{\text{сельское хозяйство}} = \{A\}$, $\mathcal{D}_{\text{услуги}} = \{F, \dots, S\}$.

Все значения share обрезались от 0 до 1.

Получаем переменные:

Общественное питание_share	Продовольствие_share	share_industry	share_services
urban_share	Транспорт_share	share_agriculture	Маркетплейсы_share
population	Здоровье_share		

Ко всем переменным в конце был применён StandardScaler для приведения в единый масштаб.

В итоге получили 1094 МО, 24 месяца, 10 переменных.

Проверим, что все переменные обладают самостоятельной объясняющей способностью и не являются избыточными. Для этого проверим корреляцию наших переменных:

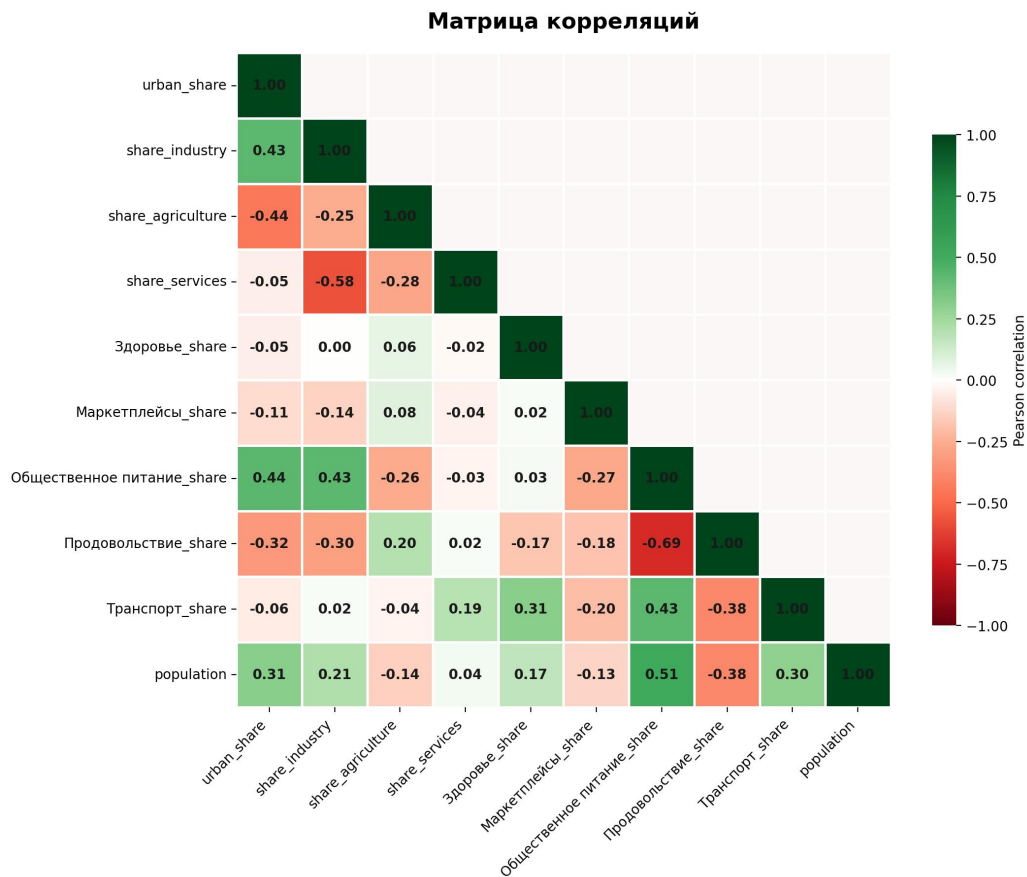


Рис. 1: Корреляция наших переменных

Также посчитаем VIF для проверки отсутствия мультиколлинеарности:

$$VIF_j = \frac{1}{1 - R_j^2},$$

где R_j^2 — коэффициент детерминации линейной регрессии переменной x_j на всех остальных переменных.

Таблица 1: Значения VIF

variable	VIF
Общественное питание_share	3.608260
Продовольствие_share	2.934186
share_industry	2.676028
share_services	2.348818
urban_share	1.702487
Транспорт_share	1.673553
share_agriculture	1.656910
Маркетплейсы_share	1.596090
population	1.434704
Здоровье_share	1.202736

Везде $VIF < 5$, мультиколлинеарность отсутствует.

4 Метрики

1. Silhouette Width (SW)

Сравнивает близость объекта к своему кластеру и к ближайшему чужому:

$$a(i) = \frac{1}{|C_a| - 1} \sum_{\substack{j \in C_a \\ j \neq i}} \|x_i - x_j\|, \quad b(i) = \min_{C_b \neq C_a} \frac{1}{|C_b|} \sum_{j \in C_b} \|x_i - x_j\|,$$

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}}, \quad SW = \frac{1}{N} \sum_{i=1}^N s(i).$$

Диапазон: $[-1, 1]$, больше — лучше. Отражает плотность и разделимость кластеров.

2. Calinski–Harabasz (CH)

Отношение межкластерной дисперсии к внутрикластерной:

$$CH = \frac{\text{tr}(B_k)/(k-1)}{\text{tr}(W_k)/(N-k)},$$

$$B_k = \sum_a |C_a|(\mu_a - \mu)(\mu_a - \mu)^\top, \quad W_k = \sum_a \sum_{x_i \in C_a} (x_i - \mu_a)(x_i - \mu_a)^\top.$$

3. S_Dbw

$$Scat = \frac{1}{k} \sum_a \frac{\|\sigma_a\|}{\|\sigma_X\|}, \quad dens(x, I) = |\{i \in I : \|x_i - x\| \leq \varepsilon\}|, \quad \varepsilon = \|\sigma_X\|,$$

$$Dens_{bw} = \frac{1}{\binom{k}{2}} \sum_{a < b} \frac{dens(\frac{\mu_a + \mu_b}{2}, I_a \cup I_b)}{\max\{dens(\mu_a, I_a), dens(\mu_b, I_b)\}}, \quad S_Dbw = Scat + Dens_{bw}.$$

Меньше — лучше. Штрафует большой разброс и высокую плотность объектов между кластерами.

4. AVI (Average Internal Connectivity)

$$AVI_a = \frac{internal_a}{internal_a + external_a}, \quad AVI = \frac{1}{k} \sum_a AVI_a,$$

$$internal_a = \sum_{i,j \in C_a} A_{ij}, \quad external_a = \sum_{i \in C_a} \sum_{j \notin C_a} A_{ij}.$$

Диапазон: $[0, 1]$, больше — лучше. Характеризует внутреннюю связность кластеров в графе.

5. AVU (Average Uncut)

$$AVU_{ab} = \frac{between_{ab}}{ext_a + ext_b - between_{ab}}, \quad AVU = \frac{1}{k} \sum_a \sum_{b \neq a} AVU_{ab},$$

$$between_{ab} = \sum_{i \in C_a} \sum_{j \in C_b} A_{ij}.$$

Меньше — лучше. Отражает долю межкластерных связей относительно внешних связей кластеров.

6. MQ (Modularity-based Quality)

$$intra_a = \frac{\sum_{i,j \in C_a} A_{ij}}{2|C_a|(|C_a| - 1)/2}, \quad inter_{ab} = \frac{\sum_{i \in C_a} \sum_{j \in C_b} A_{ij}}{|C_a||C_b|},$$

$$MQ = \frac{1}{k} \sum_a intra_a - \frac{2}{k(k-1)} \sum_{a < b} inter_{ab}.$$

Диапазон: $[-1, 1]$, больше — лучше. Высокие значения соответствуют плотным кластерам и слабым межкластерным связям.

5 Выбор метода

Будем сравнивать методы по средним метрикам месячной кластеризации. В качестве алгоритмов выбрали два самых популярных – Louvain и Leiden. Для них также перебираем resolution по сетке $[0.9, 1, 1.1]$ – параметр, отвечающий за размер выделяемых сообществ. Для проведения ребер используем cosine similarity, adaptive_rbf, tanimoto, mutual_knn. Результат экспериментов:

Для Louvain и Leiden используется максимизация модульности Q :

$$Q = \frac{1}{2m} \sum_{i,j} \left(A_{ij} - \frac{k_i k_j}{2m} \right) \mathbf{1}[c_i = c_j], \quad m = \frac{1}{2} \sum_{i,j} A_{ij}.$$

Для построения ребер используются следующие меры:

$$\cos(x_i, x_j) = \frac{x_i^\top x_j}{\|x_i\| \|x_j\|},$$

$$w_{ij}^{\text{RBF}} = \exp\left(-\frac{\|x_i - x_j\|^2}{2\sigma_i \sigma_j}\right), \quad \sigma_i = d(x_i, x_{i,(q)}),$$

$$T(x_i, x_j) = \frac{x_i^\top x_j}{\|x_i\|^2 + \|x_j\|^2 - x_i^\top x_j},$$

$$A_{ij}^{\text{mutual_knn}} = \mathbf{1}\{j \in N_q(i)\} \mathbf{1}\{i \in N_q(j)\}.$$

Ребра проводились по 75 перцентилю, а для mutual_knn — первые 250 ребер.

Результат экспериментов:

Таблица 2: Результат экспериментов

algorithm	method	resolution	k	SW	CH	S_Dbw	AVI	AVU	MQ
leiden	adaptive_rbf	0.9	5.42	0.1355	0.1676	1.9729	0.6857	0.5198	0.0841
leiden	adaptive_rbf	1.0	5.88	0.1211	0.1578	2.0371	0.6569	0.5252	0.0845
leiden	adaptive_rbf	1.1	7.00	0.1150	0.1476	2.0501	0.6155	0.5138	0.0796
leiden	cosine	0.9	3.62	0.1493	0.1872	2.2128	0.7618	0.6371	0.2585
leiden	cosine	1.0	4.12	0.1491	0.1792	2.1980	0.7277	0.6405	0.2689
leiden	cosine	1.1	4.71	0.1429	0.1703	2.1472	0.6860	0.6186	0.2841
leiden	mutual_knn	0.9	3.67	0.1420	0.1852	2.2233	0.7240	0.6164	0.1931
leiden	mutual_knn	1.0	4.25	0.1371	0.1711	2.1544	0.6796	0.5905	0.2116
leiden	mutual_knn	1.1	4.62	0.1281	0.1656	2.1483	0.6550	0.5809	0.2182
leiden	tanimoto	0.9	7.88	0.1498	0.0920	0.8762	0.6813	0.2702	0.3471
leiden	tanimoto	1.0	8.08	0.1404	0.0910	0.9055	0.6575	0.2642	0.3535
leiden	tanimoto	1.1	10.71	-0.0679	0.0706	1.0204	0.4266	0.2224	0.2462
louvain	adaptive_rbf	0.9	6.46	0.1317	0.1552	1.8798	0.6605	0.4947	0.1093
louvain	adaptive_rbf	1.0	6.12	0.1204	0.1542	2.0213	0.6476	0.5182	0.0842
louvain	adaptive_rbf	1.1	6.83	0.1145	0.1467	1.9580	0.6295	0.5008	0.0983
louvain	cosine	0.9	4.92	0.1544	0.1641	2.0526	0.6727	0.5854	0.3000
louvain	cosine	1.0	4.25	0.1485	0.1711	2.1574	0.7122	0.6274	0.2801
louvain	cosine	1.1	4.71	0.1488	0.1700	2.1638	0.6879	0.6137	0.2918
louvain	mutual_knn	0.9	4.83	0.1461	0.1632	1.9988	0.6536	0.5403	0.2446
louvain	mutual_knn	1.0	4.29	0.1345	0.1675	2.1677	0.6731	0.5895	0.2122
louvain	mutual_knn	1.1	4.79	0.1225	0.1585	2.1405	0.6335	0.5650	0.2217
louvain	tanimoto	0.9	8.83	0.1491	0.0896	0.8812	0.6355	0.2581	0.3687
louvain	tanimoto	1.0	8.12	0.1415	0.0923	0.9078	0.6541	0.2669	0.3524
louvain	tanimoto	1.1	10.08	0.0071	0.0791	1.0042	0.4737	0.2406	0.3226

Tanimoto даёт достаточно хорошие результаты, однако число кластеров у него слишком большое и это экономически неинтерпретируемо. Пожертвуем качеством ради интерпретируемости и не будем рассматривать этот способ построения ребер.

Корреляции (Спирмен) между метриками достаточно большие:

Таблица 3: Корреляции (Спирмен) между метриками

	SW	CH	S_Dbw	AVI	AVU	MQ
SW	1.000	0.900	0.767	0.133	0.933	0.817
CH	0.900	1.000	0.900	0.183	0.817	0.583
S_Dbw	0.767	0.900	1.000	-0.050	0.800	0.550
AVI	0.133	0.183	-0.050	1.000	0.000	-0.283
AVU	0.933	0.817	0.800	0.000	1.000	0.883
MQ	0.817	0.583	0.550	-0.283	0.883	1.000

Поэтому плохо будет использовать обычный Borda Count для определения лучшего метода, т.к. некоторые метрики отвечают почти за одно и то же. Исполь-

зуем метод CRITIC, который учитывает различающую способность критериев. CRITIC назначает больший вес метрикам с большей дисперсией и меньшей корреляцией с остальными, что позволяет избежать двойного учёта коррелированных метрик.

$$C_j = \sigma_j \sum_k (1 - r_{jk}), \quad w_j = \frac{C_j}{\sum_j C_j}.$$

Таблица 4: Итоговые веса

Метрика	Вес
AVU	0.258
MQ	0.168
CH	0.130
S_Dbw	0.199
SW	0.131
AVI	0.114

Результаты:

Для каждой конфигурации итоговый показатель рассчитывается как взвешенная сумма Borda-оценок по метрикам:

$$B_i = \sum_{j=1}^p w_j b_{ij},$$

где w_j — вес метрики, а b_{ij} — Borda-оценка конфигурации i по метрике j .

Таблица 5: Результаты

	algorithm	method	resolution	Borda_score
0	louvain	cosine	0.9	10.963570
1	louvain	adaptive_rbf	0.9	10.584393
2	louvain	mutual_knn	0.9	10.282420
3	leiden	adaptive_rbf	0.9	10.107124
4	louvain	cosine	1.1	9.521770
5	leiden	cosine	1.1	9.254595
6	louvain	cosine	1.0	8.981410
7	leiden	cosine	0.9	8.716075
8	leiden	cosine	1.0	8.320641
9	louvain	adaptive_rbf	1.1	8.098480
10	leiden	mutual_knn	1.0	7.924817
11	leiden	adaptive_rbf	1.0	7.584578
12	leiden	mutual_knn	1.1	7.491130
13	louvain	mutual_knn	1.1	7.453220
14	louvain	adaptive_rbf	1.0	7.399101
15	leiden	mutual_knn	0.9	7.137013
16	louvain	mutual_knn	1.0	6.859490
17	leiden	adaptive_rbf	1.1	6.320174

Победил Louvain с resolution=0.9 и методом проведения ребер cosine similarity.

6 Динамика

Кластеризуем каждый из 24 месяцев. Для того, чтобы номера кластеров совпадали в разных месяцах, применяем алгоритм Hungarian matching. Применяем классический Hungarian matching к прямоугольной матрице расстояний (допускается разное число кластеров), а для кластеров, оставшихся без пары, достраиваем соответствие по ближайшему соседу.

Сопоставление ведётся **к первому месяцу** (2023-01). Пусть $\mathcal{P}^{(1)} = \{\mu_1^{(1)}, \dots, \mu_{k_1}^{(1)}\}$ — множество центроидов кластеров первого месяца, а $\mathcal{P}^{(t)} = \{\mu_1^{(t)}, \dots, \mu_{k_t}^{(t)}\}$ — в месяц t . Сначала оба множества центроидов стандартизируются: для каждого месяца scaler обучается отдельно (свой StandardScaler для каждого меся-

ца/этапа):

$$\tilde{\mu}_a^{(t)} = \frac{\mu_a^{(t)} - \bar{\mu}^{(t)}}{\sigma^{(t)}}, \quad \tilde{\mu}_b^{(1)} = \frac{\mu_b^{(1)} - \bar{\mu}^{(1)}}{\sigma^{(1)}}.$$

Затем строится матрица попарных евклидовых расстояний между центроидами первого месяца и текущего месяца

$$D_{ab} = \|\tilde{\mu}_a^{(1)} - \tilde{\mu}_b^{(t)}\|_2,$$

и решается задача о назначениях

$$\min_{\pi} \sum_a D_{a, \pi(a)},$$

где π — инъективное отображение. Для этого применяется `linear_sum_assignment` из `scipy.optimize`, который находит оптимальное назначение для $\min(k_1, k_t)$ пар. Если $k_t > k_1$, то для каждого нераспределённого кластера из месяца t находим ближайший кластер первого месяца:

$$\pi(b) = \arg \min_a D_{ab}.$$

Если $k_1 > k_t$, лишние кластеры первого месяца не получают пары (что допустимо, так как мы сопоставляем к первому месяцу и не создаём новых кластеров).

Matching происходит к первому месяцу (2023-01), если кластеров больше, то каждый из них все равно заменяем на ближайший, новых кластеров не создаём для удобства экономической интерпретации. Также строим кластеризации отдельно по 2023 и 2024 года, чтобы посмотреть на долгосрочную динамику.

Для того, чтобы понять, стабильны ли кластеры по времени, считаем расстояние между сопоставленными кластерами по соседним месяцам (если бы считали расстояние до 2023-01, то расстояние бы линейно увеличивалось, т.к. все величины имеют какой-то тренд и со временем расстояние накапливается).

После канонизации индексов кластеров вычисляется расстояние между профилями соседних месяцев. Профили стандартизуются: для каждого месяца `scaler` обучается отдельно, после чего считается среднее расстояние по диагонали матрицы попарных расстояний:

$$d^{(t)} = \frac{1}{k} \sum_{a=1}^k \|\tilde{\mu}_a^{(t)} - \tilde{\mu}_a^{(t+1)}\|_2,$$

где $k = \min(k_t, k_{t+1})$ — число сопоставленных кластеров, а $\tilde{\mu}_a^{(t)}$ — стандартизованный центроид кластера a в месяц t после канонизации.

Итоговые расстояния:

Таблица 6: Итоговые расстояния

month_prev	month_curr	mean_distance
(2023, 1)	(2023, 2)	0.975841
(2023, 2)	(2023, 3)	0.691571
(2023, 3)	(2023, 4)	2.506343
(2023, 4)	(2023, 5)	1.956631
(2023, 5)	(2023, 6)	1.859516
(2023, 6)	(2023, 7)	0.821043
(2023, 7)	(2023, 8)	1.199523
(2023, 8)	(2023, 9)	2.222943
(2023, 9)	(2023, 10)	2.399091
(2023, 10)	(2023, 11)	2.377343
(2023, 11)	(2023, 12)	2.496521
(2023, 12)	(2024, 1)	2.895622
(2024, 1)	(2024, 2)	1.701178
(2024, 2)	(2024, 3)	0.807497
(2024, 3)	(2024, 4)	0.842803
(2024, 4)	(2024, 5)	1.176858
(2024, 5)	(2024, 6)	2.524975
(2024, 6)	(2024, 7)	1.097505
(2024, 7)	(2024, 8)	2.617362
(2024, 8)	(2024, 9)	2.352221
(2024, 9)	(2024, 10)	1.928828
(2024, 10)	(2024, 11)	1.847599
(2024, 11)	(2024, 12)	1.807223

Общая средняя distance: 1.7872

Расстояние между двумя случайными точками в нормированном евклидовом 10-мерном пространстве это $\sqrt{2 \cdot 10} = 4.47$, 1.79 — это 40%, то есть matching достаточно устойчив.

7 Интерпретация кластеров

Кластер 0. Крупные сервисно-промышленные МО

Это крупные по численности и урбанизированные территории с высокой долей услуг и заметной промышленной составляющей. Это достаточно развитые индустриальные регионы со сравнительно высокими зарплатами. Доля продовольствия в потреблении здесь самая низкая среди всех кластеров, а доля общественного питания — самая высокая. Это говорит о более высоком уровне

жизни по сравнению с другими кластерами. О развитости также свидетельствует самая большая доля расходов на транспорт.

Характерные представители: большую часть из них составляют МО в области крупных городов: Московская, Ленинградская, Казанская, Красноярская, Иркутская и другие области.

Кластер 1. Небольшие сервисные МО

Это регионы, которые не специализируются на промышленности или сельском хозяйстве, за счёт чего выделяются высокой долей сферы услуг. Это локально-сервисные экономики с низким или средним уровнем жизни. В структуре расходов заметно доминирует продовольствие. Кластер включает в себя как города, так и сельские местности, о чём говорит сравнительно низкая урбанизация.

Характерные представители: такие регионы встречаются на всей территории страны: Каменский муниципальный район, Беломорский муниципальный округ и др.

Кластер 2. Аграрные и менее урбанизированные МО

Этим регионам характерна повышенная роль сельского хозяйства при сравнительно низкой промышленной доле, низкая урбанизация (самая низкая среди всех кластеров) и относительно невысокий уровень зарплат. Большая часть муниципальных образований сосредоточена на юге и юго-западе страны, т.к. там наиболее благоприятный климат для ведения сельского хозяйства.

Характерные представители: МО на юге Центральной России, Черноземье.

Кластер 3. Промышленные МО

Этим МО свойственна высокая доля промышленности и высокая урбанизация. Доля услуг ниже, чем в сервисных группах, а средняя численность населения относительно велика. В эту группу входят промышленные регионы, моногорода и отдельные промышленные районы.

Характерные представители: Куйбышевский, Нефтеюганский муниципальные районы, городской округ Медногорск.

Кластер 4. Высокодоходные урбанизированные МО

Это самый урбанизированный кластер. Средняя зарплата здесь самая высокая среди всех групп — в основном за счёт северо-восточных регионов, где доходы повышены из-за высокой стоимости жизни, сурового климата и вахтовой занятости. Сельскохозяйственная составляющая минимальна, заметную роль

играют промышленность и услуги. Высокая зарплата не ведёт к такому же высокому уровню жизни: структура потребления здесь схожа с кластером 3, где уровень жизни средний. Значительная часть доходов населения уходит на компенсацию климатических и логистических издержек.

Характерные представители: северо-восток страны: Таймырский Долгано-Ненецкий муниципальный район, Охотский муниципальный округ.

8 Воспроизводимость

Louvain может быть нестабильным и выдавать разные результаты по разным седам. Поэтому кластеризуем с 6 разными random_seed и смотрим на разброс метрик. Для этого для каждой метрики и для количества кластеров считаем Coefficient of Variation по седам:

$$CV = \frac{\sigma}{|\mu|},$$

Таблица 7: Coefficient of Variation

Метрика	CV
SW	0.0249
CH	0.0218
S_Dbw	0.0076
AVI	0.0103
AVU	0.0130
MQ	0.0285
n_clusters	0.0255

Все $CV < 0.2$, так что метод стабилен